

INVESTIGACIÓN

Unidad 4

JavaScript: fundamentos del lenguaje

Actividad 1: Cuadro sinóptico y programación del lado del cliente



Alumno: Rivera Campos Axel Josue

Materia: Programación web

Docente: Irving Yair Nava Hernández

Fecha: 13 de julio de 2026

Investigación de JavaScript

Introducción

JavaScript es uno de los lenguajes de programación más utilizados en el desarrollo web de la actualidad. Surgió en 1995, creado por Brendan Eich para la empresa Netscape, con el objetivo de brindar de interactividad a las páginas HTML directamente en el navegador. Con el paso de los años se convirtió en un lenguaje estándar normalizado bajo la especificación ECMAScript y hoy se utiliza no solo en el navegador, sino también en servidores con ayuda de Node.js, aplicaciones móviles y de escritorio. Se describen las características básicas del lenguaje, sus estructuras de control, los tipos de datos que maneja, la forma de declarar funciones, el trabajo con objetos y arrays, el manejo de eventos y la manipulación del DOM (Document Object Model), y el manejo de la asincronía, para su aplicación práctica en la programación del lado del cliente.

1. Características básicas del lenguaje

JavaScript posee un conjunto de características que lo distinguen de otros lenguajes de programación y que explican en buena medida su popularidad para el desarrollo web.

1.1 Lenguaje interpretado

A diferencia de lenguajes compilados como C o Java, JavaScript no requiere un proceso de compilación previo para ejecutarse. El código se lee y se ejecuta línea a línea por un motor o intérprete, como V8 usado por Google Chrome y por Node.js o SpiderMonkey usado por Firefox. Esto permite probar cambios de forma inmediata, aunque los motores modernos también aplican técnicas de compilación "just-in-time" (JIT) para optimizar el rendimiento en tiempo de ejecución.

1.2 Tipado dinámico

En JavaScript no se declara de forma explícita el tipo de dato que almacenará una variable. El tipo se determina automáticamente en tiempo de ejecución, según el valor que se le asigne, y puede cambiar a lo largo del programa. Por ejemplo:

```
let dato = 5;      // dato es de tipo Number
dato = "hola";    // ahora dato es de tipo String
```

Esta flexibilidad agiliza la escritura de código, pero también requiere de mayor cuidado por parte del programador para evitar errores de conversiones de tipo inesperadas.

1.3 Lenguaje multiparadigma

JavaScript no obliga a seguir un único estilo de programación. Admite programación imperativa (instrucciones paso a paso), programación orientada a objetos (mediante objetos, clases y el sistema de prototipos), programación funcional (funciones como valores, funciones puras, uso de map, filter y reduce) y programación orientada a eventos (respuesta a acciones

del usuario o del sistema). Esto permite adaptar el estilo de código a las necesidades de cada proyecto.

2. Estructuras de control

Las estructuras de control permiten alterar el flujo secuencial de un programa, ya sea tomando decisiones o repitiendo bloques de instrucciones.

2.1 Estructuras condicionales

Las condicionales evalúan una expresión lógica y ejecutan un bloque de código distinto según el resultado sea verdadero o falso. Las principales son:

- `if | else if | else`: evalúan una o varias condiciones en cadena.
- `switch`: compara una misma variable contra varios valores posibles (casos).
- Operador ternario (condición ? valorSiVerdadero : valorSiFalso): forma abreviada de un `if/else` simple.

```
const mayor = (edad >= 18) ? "mayor de edad" : "menor de edad";
```

2.2 Bucles

Los bucles repiten un bloque de instrucciones mientras se cumpla una condición determinada. Entre los más utilizados en JavaScript están:

- `for`: se utiliza cuando se conoce de antemano el número de repeticiones.
- `while`: repite el bloque mientras la condición sea verdadera, evaluándola antes de cada iteración.
- `do...while`: similar al `while`, pero evalúa la condición después de ejecutar el bloque, garantizando al menos una ejecución.
- `for...of`: recorre los valores de un elemento iterable, como un array.
- `for...in`: recorre las propiedades (claves) enumerables de un objeto.

3. Tipos de datos

En JavaScript los tipos de datos se agrupan en dos grandes categorías, según cómo se almacenan y se manejan en memoria: primitivos y de referencia.

3.1 Tipos primitivos

Son valores simples e inmutables (no pueden modificarse una vez creados) que se almacenan y se copian por valor. JavaScript define siete tipos primitivos:

- `String`: cadenas de texto, por ejemplo `"Hola mundo"`.
- `Number`: valores numéricos, tanto enteros como decimales.
- `Boolean`: valores lógicos `true` o `false`.

- Undefined: valor por defecto de una variable declarada pero no inicializada.
- Null: representa la ausencia intencional de valor.
- Symbol: valores únicos e inmutables, usados frecuentemente como identificadores de propiedades.
- BigInt: números enteros de precisión arbitraria, mayores que el límite seguro de Number.

3.2 Tipos de referencia

Son estructuras más complejas y mutables, que se almacenan en memoria por referencia, es decir, la variable guarda una dirección que apunta al valor real, no el valor en sí. Los principales son Object (colecciones de pares clave-valor), Array (listas ordenadas de elementos) y Function (bloques de código reutilizables, que en JavaScript también se consideran un tipo de objeto).

4. Funciones

Las funciones son bloques de código reutilizables que realizan una tarea específica. JavaScript ofrece tres formas principales de definirlas.

4.1 Funciones declaradas

Se definen con la palabra clave function seguida de un nombre. Debido al fenómeno conocido como hoisting, este tipo de función queda disponible en todo su ámbito incluso antes de la línea donde aparece escrita en el código.

```
function saludar(nombre) {  
  return "Hola, " + nombre;  
}
```

4.2 Funciones expresadas

Se asignan a una variable como si fueran un valor más. A diferencia de las funciones declaradas, no sufren hoisting con su cuerpo, por lo que deben definirse antes de ser invocadas.

```
const saludar = function(nombre) {  
  return "Hola, " + nombre;  
};
```

4.3 Funciones flecha (arrow functions)

Introducidas en ECMAScript 6 (ES6), ofrecen una sintaxis más corta y no poseen su propio valor de this, sino que heredan el del contexto donde fueron creadas. Son muy utilizadas en funciones cortas y en callbacks.

```
const saludar = (nombre) => "Hola, " + nombre;
```

5. Objetos y arrays

5.1 Objetos

Un objeto es una colección de pares clave-valor que permite representar entidades del mundo real de forma estructurada. Cada clave (o propiedad) puede almacenar cualquier tipo de dato, incluyendo otros objetos o funciones, llamadas métodos cuando pertenecen a un objeto.

```
const persona = {  
  nombre: "Ana",  
  edad: 25,  
  saludar() { return "Hola"; }  
};
```

5.2 Arrays

Un array es una lista ordenada de elementos, indexada a partir de cero. Puede contener valores de distintos tipos y su tamaño se ajusta de forma dinámica.

```
const numeros = [1, 2, 3, 4];
```

JavaScript incluye métodos integrados que facilitan recorrer y transformar arrays sin necesidad de escribir bucles explícitos, entre ellos `map()` (transforma cada elemento y genera un nuevo array), `filter()` (genera un nuevo array solo con los elementos que cumplen una condición), `forEach()` (ejecuta una función por cada elemento) y `reduce()` (combina todos los elementos en un único valor).

6. Eventos y manipulación del DOM

El DOM (Document Object Model) es la representación en forma de árbol que el navegador construye a partir del documento HTML, y que JavaScript puede leer y modificar en tiempo real.

6.1 Eventos

Los eventos son acciones que ocurren en la página, ya sea generadas por el usuario (clic, envío de un formulario, pulsación de una tecla) o por el propio navegador (carga de la página). JavaScript permite "escuchar" estos eventos y ejecutar código en respuesta a ellos. Algunos de los más comunes son `click`, `submit`, `keyup` y `load`.

6.2 Manipulación del DOM

A través de distintos métodos, JavaScript puede seleccionar, crear, modificar o eliminar elementos HTML de manera dinámica, sin necesidad de recargar la página. Entre los métodos más utilizados están `getElementById()` y `querySelector()` para seleccionar elementos, `addEventListener()` para asociar un evento a un elemento y `createElement()` para crear nuevos elementos.

```
const boton = document.querySelector("#miBoton");  
boton.addEventListener("click", () => alert("¡Diste clic!"));
```

7. Asincronía

JavaScript es un lenguaje de un solo hilo de ejecución, lo que significa que solo puede realizar una tarea a la vez. Sin embargo, gracias a su modelo asíncrono, puede iniciar tareas que tardan tiempo en completarse (como peticiones a un servidor) sin bloquear el resto del programa mientras espera el resultado.

7.1 Callbacks

Un callback es una función que se pasa como argumento a otra función y que se ejecuta al finalizar una tarea, generalmente una tarea asíncrona. Fue el primer mecanismo utilizado para manejar la asincronía en JavaScript, aunque el anidamiento excesivo de callbacks puede dificultar la lectura del código (problema conocido como "callback hell").

```
setTimeout(() => console.log("Han pasado 2 segundos"), 2000);
```

7.2 Promesas (Promises)

Una promesa es un objeto que representa el resultado eventual (éxito o fallo) de una operación asíncrona. Ofrece los métodos `then()` para manejar el resultado exitoso y `catch()` para manejar errores, y permite encadenar operaciones de forma más ordenada que los callbacks anidados.

```
fetch("https://api.ejemplo.com/datos")  
  .then(respuesta => respuesta.json())  
  .catch(error => console.error(error));
```

7.3 Async / await

Introducida en ES2017, esta sintaxis se construye sobre las promesas y permite escribir código asíncrono con una apariencia secuencial, más fácil de leer y depurar. La palabra clave `async` convierte una función en asíncrona, y `await` pausa su ejecución hasta que la promesa correspondiente se resuelva.

```
async function obtenerDatos() {  
  const respuesta = await fetch("https://api.ejemplo.com/datos");  
  const datos = await respuesta.json();  
  return datos;  
}
```

Conclusión

JavaScript es un lenguaje flexible y en constante evolución, cuyas características básicas son: ser interpretado, de tipado dinámico y multiparadigma, lo hacen accesible para comenzar a programar, a la vez que ofrece herramientas como estructuras de control, un sistema de tipos claro, distintas formas de definir funciones, manejo de objetos y arrays, interacción con el DOM y mecanismos modernos para trabajar con asincronía, que permiten construir aplicaciones web interactivas y complejas. Comprender estos fundamentos resulta indispensable para cualquier persona que desee especializarse en el desarrollo web del lado del cliente.

JAVASCRIPT

Lenguaje de programación

1 Características básicas

Interpretado

Se ejecuta línea a línea mediante un intérprete (navegador o Node.js), sin compilarse antes. Ej: motor V8, Node.js

Tipado dinámico

El tipo de una variable se define en tiempo de ejecución y puede cambiar durante el programa. Ej: let x = 5; x = "hola";

Multiparadigma

Combina distintos estilos de programación según la necesidad del proyecto. Ej: POO, funcional, imperativa, eventos

2 Estructuras de control

Condicionales

Ejecutan bloques de código distintos según si una condición lógica es verdadera o falsa. Ej: if, else if, else, switch

Bucles

Repiten un bloque de instrucciones mientras se cumpla una condición. Ej: for, while, do-while, for...of

3 Tipos de datos

Primitivos

Valores simples e inmutables, se almacenan y copian por valor. Ej: String, Number, Boolean, Undefined, Null, Symbol, BigInt

De referencia

Estructuras complejas y mutables, se pasan por referencia en memoria. Ej: Object, Array, Function

4 Funciones

Declaradas

Se definen con la palabra clave "function"; sufren hoisting (se elevan antes de ejecutarse). Ej: function saludar() {}

Expresadas

Se asignan a una variable; no sufren hoisting. Ej: const saludar = function() {}

Arrow functions

Sintaxis corta de ES6; no poseen su propio valor de "this". Ej: const saludar = () => {}

5 Objetos y arrays

Objetos

Colecciones de pares clave-valor que representan entidades. Ej: { nombre: "Ana", edad: 25 }

Arrays

Listas ordenadas de elementos, indexadas desde 0. Ej: [1, 2, 3]

Métodos de arrays

Funciones integradas para recorrer y transformar colecciones. Ej: map, filter, forEach, reduce

6 Eventos y DOM

Eventos

Acciones del usuario o del navegador que el código puede detectar y responder. Ej: click, submit, keyup, load

Manipulación del DOM

Métodos para seleccionar, crear y modificar elementos HTML dinámicamente. Ej: getElementById, querySelector, addEventListener

7 Asíncrona

Callbacks

Función que se pasa como argumento y se ejecuta al terminar una tarea asíncrona. Ej: setTimeout(fn, 1000)

Promesas

Objeto que representa el resultado futuro (éxito o fallo) de una operación asíncrona. Ej: fetch(url).then().catch()

Async / await

Sintaxis moderna para escribir código asíncrono con apariencia secuencial. Ej: async function f(){ await fetch(url); }